

1: Create a simple HTML webpage that serves as a personal homepage. The webpage should include a main heading with your name, a short paragraph introducing yourself, and a bulleted list of three hobbies or favorite activities. Additionally, add a hyperlink to a website you like and an image that represents one of your hobbies. Save the file as homepage.html.

Source Code:

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4    <meta charset="UTF-8">
5    <meta name="viewport" content="width=device-width, initial-scale=1.0">
6    <title>My Homepage</title>
7  </head>
8  <body>
9    <h1>ABSAR</h1>
10   <p>Hello! My name is Absar, and I am passionate about learning new skills and exploring
    the world around me. Welcome to my personal homepage!</p>
11
12   <h2>My Hobbies</h2>
13   <ul>
14     <li>Reading science fiction books</li>
15     <li>Traveling to new places</li>
16     <li>Photography</li>
17   </ul>
18
19   <h2>Favorite Website</h2>
20   <p>Check out this awesome website: <a href="https://www.youtube.com/"
    target="_blank">Example Website</a></p>
21
22   <h2>Photography</h2>
23   
```

OUTPUT:

ABSAR

Hello! My name is Absar, and I am passionate about learning new skills and exploring the world around me. Welcome to my personal homepage!

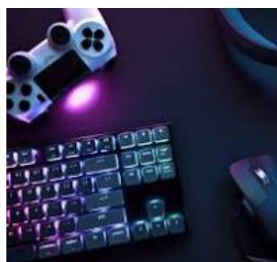
My Hobbies

- Reading science fiction books
- Traveling to new places
- Photography

Favorite Website

Check out this awesome website: [Example Website](#)

Photography



2:: Design a basic personal portfolio website consisting of two HTML pages. The first page should include a welcome message, a brief introduction, and a list of your top three favorite books or movies. The second page should contain a small gallery with three images of your choice and a heading above each image to describe it. Save the pages as index.html and gallery.html, and store them in a folder named PortfolioWebsite.

Source Code:

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Welcome to My Portfolio</title>
7  </head>
8  <body>
9      <h1>Welcome to My Portfolio</h1>
10     <p>Hi, I am Absar! I am a creative individual with a passion for learning and growing.
      This is my personal portfolio, where I share my favorite things and hobbies.</p>
11
12     <h2>My Top 3 Favorite Books/Movies</h2>
13     <ul>
14         <li>The Alchemist</li>
15         <li>Inception</li>
16         <li>Harry Potter</li>
17     </ul>
18     <p>Explore my gallery: <a href="gallery.html">Go to Gallery</a></p>
19 </body>
20 </html>
```

OUTPUT:

Welcome to My Portfolio

Hi, I am Absar! I am a creative individual with a passion for learning and growing. This is my personal portfolio, where I share my favorite things and hobbies.

My Top 3 Favorite Books/Movies

- The Alchemist
- Inception
- Harry Potter

Explore my gallery: [Go to Gallery](#).

Gallery

Photo 1: Beautiful Sunset



Photo 2: Mountain Adventure



Photo 3: Cityscape at Night



[Back to Home](#)

LAB 2

1: Create a webpage that includes a timetable of your weekly classes using an HTML table. The table should have headings for each day of the week and rows for each time slot. Also, include a registration form below the timetable where users can enter their name, email, and select their class timing from a dropdown.

Source Code:

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Weekly Class Timetable</title>
7  </head>
8  <body>
9      <h1>Weekly Class Timetable</h1>
10     <table>
11         <thead>
12             <tr>
13                 <th>Time Slot</th>
14                 <th>Monday</th>
15                 <th>Tuesday</th>
16                 <th>Wednesday</th>
17                 <th>Thursday</th>
18                 <th>Friday</th>
19             </tr>
20         </thead>
21         <tbody>
22             <tr>
23                 <td>9:00 AM - 10:00 AM</td>
24                 <td>Math</td>
25                 <td>Physics</td>
26                 <td>Math</td>
27                 <td>Chemistry</td>
28                 <td>Biology</td>
29             </tr>
30             <tr>
31                 <td>10:00 AM - 11:00 AM</td>
32                 <td>English</td>
33                 <td>Computer Science</td>
34                 <td>English</td>
35                 <td>Physics</td>
36                 <td>Chemistry</td>
37             </tr>
38             <tr>
39                 <td>11:00 AM - 12:00 PM</td>
40                 <td>Computer Science</td>
41                 <td>Math</td>
42                 <td>Physics</td>
43                 <td>English</td>
44                 <td>Math</td>
45             </tr>
46             <tr>
47                 <td>12:00 PM - 1:00 PM</td>
48                 <td>Chemistry</td>
49                 <td>Biology</td>
50                 <td>Chemistry</td>
51                 <td>Math</td>
52                 <td>Physics</td>
53             </tr>
54         </tbody>
55     </table>
56
57     <h2>Register for a Class</h2>
58     <form action="#" method="POST">
59         <div>
60             <label for="name">Name:</label>
61             <input type="text" id="name" name="name" required>
62         </div>
63         <div>
64             <label for="email">Email:</label>
65             <input type="email" id="email" name="email" required>
66         </div>
67         <br>
68         <div>
69             <label for="timing">Select Class Timing:</label>
70             <select id="timing" name="timing" required>
71                 <option value="9:00 AM - 10:00 AM">9:00 AM - 10:00 AM</option>
72                 <option value="10:00 AM - 11:00 AM">10:00 AM - 11:00 AM</option>

```

```
70         <select id="timing" name="timing" required>
71             <option value="9:00 AM - 10:00 AM">9:00 AM - 10:00 AM</option>
72             <option value="10:00 AM - 11:00 AM">10:00 AM - 11:00 AM</option>
73             <option value="11:00 AM - 12:00 PM">11:00 AM - 12:00 PM</option>
74             <option value="12:00 PM - 1:00 PM">12:00 PM - 1:00 PM</option>
75         </select>
76     </div>
77     <div>
78         <button type="submit">Register</button>
79     </div>
80 </form>
81 </body>
82 </html>
```

OUTPUT:

Weekly Class Timetable

Time Slot	Monday	Tuesday	Wednesday	Thursday	Friday
9:00 AM - 10:00 AM	Math	Physics	Math	Chemistry	Biology
10:00 AM - 11:00 AM	English	Computer Science	English	Physics	Chemistry
11:00 AM - 12:00 PM	Computer Science	Math	Physics	English	Math
12:00 PM - 1:00 PM	Chemistry	Biology	Chemistry	Math	Physics

Register for a Class

Name:

Email:

Select Class Timing:

2: Design a basic feedback form using HTML. The form should include fields for name, email, feedback message (textarea), and a rating system (radio buttons). Add a submit button at the end and save the file as feedback.html.

Source Code:

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Feedback Form</title>
7  </head>
8  <body>
9      <h1>Feedback Form</h1>
10     <form action="#" method="POST">
11         <div>
12             <label for="name">Name:</label>
13             <input type="text" id="name" name="name" required>
14         </div>
15         <div>
16             <label for="email">Email:</label>
17             <input type="email" id="email" name="email" required>
18         </div>
19         <br>
20         <div>
21             <p>Feedback</p>
22             <textarea id="message" name="message" rows="5" required></textarea>
23         </div>
24         <br>
25         <div>
26             <label>Rating:</label>
27             <div>
28                 <input type="radio" id="rating1" name="rating" value="1" required>
29                 <label for="rating1">1 - Poor</label>
30             </div>
31             <div>
32                 <input type="radio" id="rating2" name="rating" value="2">
33                 <label for="rating2">2 - Fair</label>
34             </div>
35             <div>
36                 <input type="radio" id="rating3" name="rating" value="3">
37                 <label for="rating3">3 - Good</label>
38             </div>
39             <div>
```

```
40         <input type="radio" id="rating4" name="rating" value="4">
41         <label for="rating4">4 - Very Good</label>
42     </div>
43     <div>
44         <input type="radio" id="rating5" name="rating" value="5">
45         <label for="rating5">5 - Excellent</label>
46     </div>
47 </div>
48 <br>
49 <div>
50     <button type="submit">Submit Feedback</button>
51 </div>
52 </form>
53 </body>
54 </html>
55
```

Output:

Feedback Form

Name:

Email:

Feedback

Rating:

- ☐ 1 - Poor
- ☐ 2 - Fair
- ☐ 3 - Good
- ☐ 4 - Very Good
- ☐ 5 - Excellent

1: Design a homepage for a university that includes:

- A main heading styled using internal CSS to display the university name prominently.
- A navigation bar for links like "Home," "About Us," "Departments," and "Contact Us," styled using external CSS.
- A welcome paragraph styled using inline CSS to set a custom font size and background color.

HTML Code:

```
<> TASK1.html > ...
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>University Homepage</title>
7      <link rel="stylesheet" href="styles.css">
8      <style>
9          h1 {
10             font-family: Arial, sans-serif;
11             color: #004080;
12             text-align: center;
13             margin-top: 20px;
14         }
15     </style>
16 </head>
17 <body>
18     <h1>Welcome to XYZ University</h1>
19     <nav>
20         <a href="#home">Home</a>
21         <a href="#about">About Us</a>
22         <a href="#departments">Departments</a>
23         <a href="#contact">Contact Us</a>
24     </nav>
25     <p style="font-size: 18px; background-color: #f0f8ff; padding: 10px; text-align: justify;">
26         Welcome to XYZ University, a premier institution dedicated to academic excellence and innovation.
27         We are proud to offer a wide range of programs and departments to help students achieve their
28         educational and professional goals.
29     </p>
30 </body>
31 </html>
```

CSS CODE:

```
# styles.css > nav a
1  nav {
2      background-color: #004080;
3      padding: 10px;
4      text-align: center;
5  }
6  nav a {
7      color: white;
8      text-decoration: none;
9      margin: 0 15px;
10     font-weight: bold;
11 }
12 nav a:hover {
13     text-decoration: underline;
14 }
15
```

OUTPUT:

Welcome to XYZ University

[Home](#) [About Us](#) [Departments](#) [Contact Us](#)

Welcome to XYZ University, a premier institution dedicated to academic excellence and innovation. We are proud to offer a wide range of programs and departments to help students achieve their educational and professional goals.

2: Create a personal portfolio webpage that includes:

- A main heading with your name, styled using an ID.
- A section for your hobbies, each with a subheading styled using classes.
- A list of your skills (ordered or unordered), with custom styles applied using an external stylesheet.

HTML Code:

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Personal Portfolio</title>
7      <link rel="stylesheet" href="styles.css">
8  </head>
9  <body>
10     <h1 id="name-heading">John Doe</h1>
11     <section>
12         <h2 class="hobby-heading">Hobbies</h2>
13         <ul>
14             <li>Reading Books</li>
15             <li>Photography</li>
16             <li>Traveling</li>
17         </ul>
18     </section>
19     <section>
20         <h2 class="hobby-heading">Skills</h2>
21         <ul class="skills-list">
22             <li>Web Development</li>
23             <li>JavaScript Programming</li>
24             <li>Database Management</li>
25         </ul>
26     </section>
27 </body>
28 </html>
```

CSS CODE:

```
1  #name-heading {
2      font-family: Arial, sans-serif;
3      color: #004080;
4      text-align: center;
5      margin-top: 20px;
6      font-size: 2em;
7  }
8  .hobby-heading {
9      font-family: Verdana, sans-serif;
10     color: #333;
11     font-size: 1.5em;
12     margin-bottom: 10px;
13 }
14 .skills-list {
15     font-family: Georgia, serif;
16     color: #444;
17     list-style-type: square;
18     margin: 20px;
19     padding: 0;
20 }
21 .skills-list li {
22     margin-bottom: 5px;
23 }
```

Output:

John Doe

Hobbies

- Reading Books
- Photography
- Traveling

Skills

- Web Development
 - JavaScript Programming
 - Database Management
-

LAB 4

1: Design a **product landing page** that includes:

1. A favicon for the page.
2. A header, content section, and footer styled using **divs** and the **Box Model**.
3. Use **relative** positioning to adjust the content placement and **absolute** positioning for a floating "Contact Us" button.
4. Style the typography of the page (e.g., font family, size, and line spacing).

HTML CODE:

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Product Landing Page</title>
7      <link rel="stylesheet" href="styles1.css"> <!-- External CSS -->
8  </head>
9  <body>
10 <header>
11     <h1>Welcome to Our Product</h1>
12     <p>Your solution to an amazing experience</p>
13 </header>
14 <div class="content">
15     <h2>About Our Product</h2>
16     <p>
17         Our product is designed to make your life easier and more efficient. With cutting-edge
18         technology, it offers seamless integration and user-friendly features.
19     </p>
20     <p>
21         Benefits include improved productivity, enhanced security, and exceptional support.
22         Whether you're a professional or just starting out, our product is tailored for everyone.
23     </p>
24     <br>
25     <button class="contact-us">Contact Us</button>
26 </div>
27 <footer>
28     <p>&copy; 2024 Our Product. All rights reserved.</p>
29 </footer>
30 </body>
31 </html>
```

CSS CODE:

```
1  body {
2      font-family: Arial, sans-serif;
3      line-height: 1.6;
4      margin: 0;
5      padding: 0;
6  }
7  header {
8      background-color: #004080;
9      color: white;
10     padding: 20px;
11     text-align: center;
12     border-bottom: 5px solid #003366;
13 }
14 .content {
15     padding: 20px;
16     margin: 20px auto;
17     width: 80%;
18     border: 1px solid #ddd;
19     box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
20     background-color: #f9f9f9;
21     position: relative;
22 }
23 footer {
24     background-color: #004080;
25     color: white;
26     text-align: center;
27     padding: 10px;
28     margin-top: 20px;
29     border-top: 5px solid #003366;
30 }
```

```
31 .contact-us {
32     position: absolute;
33     bottom: 20px;
34     background-color: #ff6600;
35     color: white;
36     padding: 10px 15px;
37     border: none;
38     border-radius: 5px;
39     cursor: pointer;
40     font-size: 16px;
41     text-align: center;
42 }
43
44 .contact-us:hover {
45     background-color: #ff8533;
46 }
47
```

OUTPUT:

Welcome to Our Product

Your solution to an amazing experience

About Our Product

Our product is designed to make your life easier and more efficient. With cutting-edge technology, it offers seamless integration and user-friendly features.

Benefits include improved productivity, enhanced security, and exceptional support. Whether you're a professional or just starting out, our product is tailored for everyone.

Contact Us

© 2024 Our Product. All rights reserved.

2: Design a homepage for a book store that includes:

1. A header section with the store name styled using CSS typography.
2. A section for featured books, divided into categories using divs.
3. Use the Box Model to add padding, borders, and margins to the book sections.
4. Include a "Buy Now" button for each book, styled using CSS positioning and typography.

HTML CODE:

```
<> TASK2.HTML > html > head > link
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Bookstore Homepage</title>
7      <link rel="stylesheet" href="styles2.css">
8  </head>
9  <body>
10     <header>
11         Welcome to ABC Bookstore
12     </header>
13     <section class="featured-books">
14         <div class="book-category">
15             <h3>The Great Gatsby</h3>
16             <p>A classic novel by F. Scott Fitzgerald.</p>
17             <button class="buy-now">Buy Now</button>
18         </div>
19         <div class="book-category">
20             <h3>Sapiens: A Brief History of Humankind</h3>
21             <p>An insightful book by Yuval Noah Harari.</p>
22             <button class="buy-now">Buy Now</button>
23         </div>
24         <div class="book-category">
25             <h3>Harry Potter and the Sorcerer's Stone</h3>
26             <p>J.K. Rowling's magical world for kids.</p>
27             <button class="buy-now">Buy Now</button>
28         </div>
29     </section>
30 </body>
31 </html>
```

CSS CODE:

```
body {
  font-family: Arial, sans-serif;
  margin: 0;
  padding: 0;
  background-color: #f8f8f8;
  line-height: 1.6;
}

header {
  background-color: #2a2a72;
  color: white;
  text-align: center;
  padding: 20px;
  font-size: 2em;
  border-bottom: 5px solid #1e1e5a;
}

.featured-books {
  padding: 20px;
  margin: 20px auto;
  width: 90%;
  display: flex;
  gap: 20px;
  flex-wrap: wrap;
  justify-content: space-around;
}

.book-category {
  border: 1px solid #ddd;
  padding: 15px;
  margin: 10px;
  width: 280px;
  background-color: white;
  box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
  text-align: center;
  position: relative;
}

.book-category h3 {
  color: #2a2a72;
  font-size: 1.2em;
  margin-bottom: 10px;
}

.buy-now {
  position: absolute;
  bottom: 15px;
  left: 50%;
  transform: translateX(-50%);
  background-color: #ff6600;
  color: white;
  border: none;
  padding: 10px 20px;
  font-size: 1em;
  cursor: pointer;
  border-radius: 5px;
  text-transform: uppercase;
}

.buy-now:hover {
  background-color: #e65c00;
}
```

OUTPUT:

Welcome to ABC Bookstore

The Great Gatsby

A classic novel by F. Scott Fitzgerald.

BUY NOW

Sapiens: A Brief History of Humankind

An insightful book by Yuval Noah

BUY NOW

Harry Potter and the Sorcerer's Stone

J.K. Rowling
A magical world for kids.

BUY NOW

LAB TASK:

SOURCE CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Tech Startup</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.1/dist/css/bootstrap.min.css"
  rel="stylesheet">
</head>
<body>
  <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
    <div class="container-fluid">
      <a class="navbar-brand" href="#">Tech Startup</a>
      <button class="navbar-toggler" type="button" data-bs-
toggle="collapse" data-bs-target="#navbarNav" aria-controls="navbarNav" aria-
expanded="false" aria-label="Toggle navigation">
        <span class="navbar-toggler-icon"></span>
      </button>
      <div class="collapse navbar-collapse" id="navbarNav">
        <ul class="navbar-nav">
          <li class="nav-item">
            <a class="nav-link" href="#">Home</a>
          </li>
          <li class="nav-item">
            <a class="nav-link" href="#">Services</a>
          </li>
          <li class="nav-item">
            <a class="nav-link" href="#">Contact Us</a>
          </li>
        </ul>
      </div>
    </div>
  </nav>
  <div class="container-fluid bg-light text-center py-5">
    <h1 class="display-4">Welcome to Our Tech Startup</h1>
    <p class="lead">Innovative solutions for a smarter future</p>
    <button class="btn btn-primary btn-lg">Learn More</button>
```

```
</div>
<div class="container-fluid py-5">
  <h2 class="text-center mb-4">Our Services</h2>
  <div class="row text-center">
    <div class="col-md-4">
      <h4>Web Development</h4>
      <p>Creating modern, responsive websites.</p>
    </div>
    <div class="col-md-4">
      <h4>App Development</h4>
      <p>Custom mobile app solutions.</p>
    </div>
    <div class="col-md-4">
      <h4>AI & Machine Learning</h4>
      <p>Cutting-edge AI-driven technologies.</p>
    </div>
  </div>
</div>
</div>
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.1/dist/js/bootstrap.bundle.mi
n.js"></script>
</body>
</html>
```

OUTPUT:

Tech Startup Home Services Contact Us

Welcome to Our Tech Startup

Innovative solutions for a smarter future

[Learn More](#)

Our Services

Web Development

Creating modern, responsive websites.

App Development

Custom mobile app solutions.

AI & Machine Learning

Cutting-edge AI-driven technologies.

HOME TASK:

SOURCE CODE:

```
<!DOCTYPE html>

<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Travel Agency</title>
  <!-- Bootstrap CSS -->
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.1/dist/css/bootstrap.min.css
" rel="stylesheet">
  <!-- Font Awesome -->
  <link href="https://cdn.jsdelivr.net/npm/@fortawesome/fontawesome-free@6.0.0-
beta3/css/all.min.css" rel="stylesheet">
</head>
<body>
  <nav class="navbar navbar-expand-lg navbar-light bg-light">
    <div class="container-fluid">
      <a class="navbar-brand" href="#">Travel Agency</a>
      <button class="navbar-toggler" type="button" data-bs-
toggle="collapse" data-bs-target="#navbarNav" aria-controls="navbarNav" aria-
expanded="false" aria-label="Toggle navigation">
        <span class="navbar-toggler-icon"></span>
      </button>
      <div class="collapse navbar-collapse" id="navbarNav">
        <ul class="navbar-nav">
          <li class="nav-item">
            <a class="nav-link" href="#">Home</a>
          </li>
          <li class="nav-item">
            <a class="nav-link" href="#">Packages</a>
          </li>
          <li class="nav-item">
            <a class="nav-link" href="#">Contact Us</a>
          </li>
        </ul>
      </div>
    </div>
  </nav>
  <div class="container py-5">
    <h2 class="text-center mb-4">Featured Destinations</h2>
```

```

<div class="row">
  <div class="col-md-4 text-center">
    
    <h4>Maldives</h4>
    <p>Experience the serene beauty of the islands.</p>
  </div>
  <div class="col-md-4 text-center">
    
    <h4>Paris</h4>
    <p>Discover the charm of the City of Light.</p>
  </div>
  <div class="col-md-4 text-center">
    
    <h4>Japan</h4>
    <p>Explore the rich culture and modern marvels.</p>
  </div>
</div>
<div class="bg-light text-center py-4">
  <p><i class="fas fa-phone-alt"></i> +123 456 7890</p>
  <p><i class="fas fa-envelope"></i> info@travelagency.com</p>
</div>
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.1/dist/js/bootstrap.bundle.mi
n.js"></script>
</body>
</html>

```

OUTPUT:

Travel Agency Home Packages Contact Us

Featured Destinations



Maldives



Paris



Japan

+123 456 7890

info@travelagency.com

LAB TASK:

SOURCE CODE:

```
<!DOCTYPE html>

<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Lab Task - JavaScript Webpage</title>
  <script src="labtask.js"></script>
</head>
<body>
  <h1>JavaScript Webpage</h1>
  <h2>Form for Personalized Greeting</h2>
  <form onsubmit="event.preventDefault(); displayGreeting();">
    <label for="nameInput">Enter your name:</label>
    <input type="text" id="nameInput" placeholder="Your name">
    <button type="submit">Submit</button>
  </form>
  <p id="greetingMessage" style="font-weight: bold; color:
green;"></p>
</body>
</html>
```

JAVASCRIPT CODE:

```
window.onload = function () {

  alert("Welcome to our webpage!");
};
function displayGreeting() {
  const userName = document.getElementById("nameInput").value;
  if (userName.trim() === "") {
    alert("Please enter your name.");
  } else {
    document.getElementById("greetingMessage").innerText = `Hello,
${userName}! Welcome to our webpage.`;
  }
}
console.log("First 10 Natural Numbers:");
for (let i = 1; i <= 10; i++) {
  console.log(i);
}
```

OUTPUT:

JavaScript Webpage

Form for Personalized Greeting

Enter your name:

This page says

Welcome to our webpage!

OK

HOME TASK:

SOURCE CODE:

```
<!DOCTYPE html>

<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Home Task - Even or Odd</title>
</head>
<body>
  <h1>Even or Odd Checker</h1>
  <button onclick="checkEvenOdd()">Check Number</button>
  <script>
    // Function to check if a number is even or odd
    function checkEvenOdd() {
      const userInput = prompt("Enter a number:");
      const number = parseInt(userInput);

      // Validate the input
      if (isNaN(number)) {
        alert("Please enter a valid number.");
      } else {
        if (number % 2 === 0) {
          alert(`The number ${number} is Even.`);
        } else {
          alert(`The number ${number} is Odd.`);
        }
      }
    }
  </script>
</body>
</html>
```

OUTPUT:

Even or Odd Checker

Check Number

This page says

Enter a number:

OK Cancel

This page says

The number 24 is Even.

OK

LAB 7

LAB TASK:

SOURCE CODE:

```
<!DOCTYPE html>

<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Lab Task</title>
  <style>
    #changeButton {
      padding: 10px 20px;
      font-size: 16px;
      cursor: pointer;
      border: none;
      background-color: #007bff;
      color: white;
      border-radius: 5px;
      transition: background-color 0.3s;
    }
  </style>
</head>
<body>
  <h1>Lab Task Webpage</h1>
  <p id="content">This is a sample paragraph.</p>
  <button id="changeButton"
    onclick="changeParagraphContent()"
    onmouseover="hoverButton(this)"
    onmouseout="unhoverButton(this)">
    Click Me
  </button>
  <script>
    function changeParagraphContent() {
      const paragraph = document.getElementById("content");
      paragraph.textContent = "The content of this paragraph has
been changed!";
      const newParagraph = document.createElement("p");
      newParagraph.textContent = "This is a new paragraph added
dynamically.";
      document.body.appendChild(newParagraph);
    }
  </script>
</body>
</html>
```

```
function hoverButton(button) {  
    button.style.backgroundColor = "#0056b3";  
}  
  
function unhoverButton(button) {  
    button.style.backgroundColor = "#007bff";  
}  
</script>  
</body>  
</html>
```

OUTPUT (BEFORE CLICK):

Lab Task Webpage

This is a sample paragraph.

Click Me

OUTPUT (AFTER CLICK):

Lab Task Webpage

The content of this paragraph has been changed!

Click Me

This is a new paragraph added dynamically.

This is a new paragraph added dynamically.

This is a new paragraph added dynamically.

This is a new paragraph added dynamically.

HOME TASK:**SOURCE CODE:**

```
<!DOCTYPE html>

<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Home Task</title>
  <style>
    #contentArea {
      padding: 20px;
      background-color: #f9f9f9;
      border: 1px solid #ccc;
    }
    footer {
      text-align: center;
      padding: 10px;
      background-color: #007bff;
      color: white;
      margin-top: 20px;
    }
  </style>
</head>
<body>
  <header>
    <h1 id="headerText">Header Section</h1>
    <button onclick="updateHeaderText()">Update Header</button>
  </header>

  <section id="contentArea">
    <h2>Content Area</h2>
    <p>This is the main content area of the webpage.</p>
    <label for="colorSelector">Change Background Color:</label>
    <select id="colorSelector" onchange="changeBackgroundColor()">
      <option value="#f9f9f9">Default</option>
      <option value="#ffcccc">Light Red</option>
      <option value="#ccffcc">Light Green</option>
      <option value="#ccccff">Light Blue</option>
    </select>
  </section>
  <script>
    // Update header text dynamically
```

```
function updateHeaderText() {
    document.getElementById("headerText").textContent = "Updated
Header Text!";
}

// Change background color of content area
function changeBackgroundColor() {
    const selectedColor =
document.getElementById("colorSelector").value;
    document.getElementById("contentArea").style.backgroundColor =
selectedColor;
}

// Add a new footer dynamically after 5 seconds
window.onload = function () {
    setTimeout(() => {
        const newFooter = document.createElement("footer");
        newFooter.textContent = "This is a dynamically added
footer.";
        document.body.appendChild(newFooter);
    }, 5000);
};
</script>
</body>
</html>
```

OUTPUT (BEFORE CLICKING):

Header Section

Update Header

Content Area

This is the main content area of the webpage.

Change Background Color: Default

OUTPUT (AFTER CLICKING THE BUTTON AND CHANGING THE COLOR):

Updated Header Text!

Update Header

Content Area

This is the main content area of the webpage.

Change Background Color: Light Green

This is a dynamically added footer.

LAB TASK:

SOURCE CODE:

```
<!DOCTYPE html>

<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Lab Task - Student List</title>
  <script
src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.min.js"></script>
  <style>
    #studentSection {
      margin: 20px;
      border: 1px solid #ccc;
      padding: 10px;
      border-radius: 5px;
    }
    .student {
      margin-bottom: 10px;
    }
  </style>
</head>
<body>
  <h1>Student List</h1>
  <button id="toggleButton">Toggle Student List</button>
  <input type="text" id="searchField" placeholder="Search student by name"
style="margin-left: 10px;">
  <div id="studentSection">
    <!-- Students will be displayed here -->
  </div>
</body>
</html>
```

JAVASCRIPT CODE:

```
$(document).ready(function () {  
    // JSON data  
    const students = [  
        { name: "Alice", age: 20, grade: "A" },  
        { name: "Bob", age: 22, grade: "B" },  
        { name: "Charlie", age: 19, grade: "A+" },  
        { name: "David", age: 21, grade: "B+" },  
    ];  
  
    // Display student list  
    function displayStudents(filter = "") {  
        $("#studentSection").empty();  
        const filteredStudents = students.filter(student =>  
student.name.toLowerCase().includes(filter.toLowerCase()));  
        filteredStudents.forEach(student => {  
            $("#studentSection").append(  
                `<div class="student">  
                    <strong>Name:</strong> ${student.name}<br>  
                    <strong>Age:</strong> ${student.age}<br>  
                    <strong>Grade:</strong> ${student.grade}  
                </div>`  
            );  
        });  
        if (filteredStudents.length === 0) {  
            $("#studentSection").append("<p>No students found.</p>");  
        }  
    }  
  
    // Initial display  
    displayStudents();  
  
    // Toggle visibility  
    $("#toggleButton").click(function () {  
        $("#studentSection").fadeToggle();  
    });  
  
    // Search filter  
    $("#searchField").on("input", function () {  
        const searchText = $(this).val();  
        displayStudents(searchText);  
    });  
});
```

OUTPUT (BEFORE CLICKING TOGGLE):

Student List

Name: Alice
Age: 20
Grade: A

Name: Bob
Age: 22
Grade: B

Name: Charlie
Age: 19
Grade: A+

Name: David
Age: 21
Grade: B+

OUTPUT (AFTER CLICKING TOGGLE):

Student List

OUTPUT (SEARCHING STUDENT):

Student List

Name: Alice
Age: 20
Grade: A

HOME TASK:

SOURCE CODE:

```
<!DOCTYPE html>

<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Home Task - Product Catalog</title>
  <script
src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.min.js"></scri
pt>
  <style>
    .product {
      border: 1px solid #ccc;
      padding: 10px;
      margin: 10px;
      border-radius: 5px;
      text-align: center;
      width: 200px;
    }
    .product img {
      width: 100px;
      height: 100px;
      object-fit: cover;
    }
    .catalog {
      display: flex;
      flex-wrap: wrap;
    }
  </style>
</head>
<body>
  <h1>Product Catalog</h1>
  <div>
    <label><input type="checkbox" class="categoryFilter"
value="Electronics"> Electronics</label>
    <label><input type="checkbox" class="categoryFilter"
value="Clothing"> Clothing</label>
  </div>
  <div class="catalog" id="productCatalog">
    <!-- Products will be displayed here -->
  </div></body></html>
```

JAVASCRIPT CODE:

```
<script>
    $(document).ready(function () {
        // JSON product data
        const products = [
            { name: "Laptop", price: "$800", description: "High-
performance laptop", category: "Electronics", imageUrl: "laptop.jpg",
details: "16GB RAM, 512GB SSD" },
            { name: "Smartphone", price: "$500", description: "Latest
smartphone", category: "Electronics", imageUrl: "phone.jpg", details: "128GB
Storage, 5G" },
            { name: "T-Shirt", price: "$20", description: "Comfortable
cotton T-shirt", category: "Clothing", imageUrl: "tshirt.jpg", details:
"Available in various sizes" },
            { name: "Jeans", price: "$40", description: "Stylish blue
jeans", category: "Clothing", imageUrl: "jeans.jpg", details: "Available in
various sizes" },
        ];

        // Display products
        function displayProducts(categories = []) {
            $("#productCatalog").empty();
            const filteredProducts = categories.length === 0
                ? products
                : products.filter(product =>
categories.includes(product.category));
            filteredProducts.forEach(product => {
                $("#productCatalog").append(
                    `<div class="product">
                        
                        <h3>${product.name}</h3>
                        <p>${product.description}</p>
                        <p><strong>Price:</strong> ${product.price}</p>
                        <button class="detailsButton">Details</button>
                        <div class="productDetails" style="display:
none;">
                            <p>${product.details}</p>
                        </div>
                    </div>`;
            });
        }
    });
}
```

```
// Initial display
displayProducts();

// Filter by category
$(".categoryFilter").change(function () {
    const selectedCategories =
$(".categoryFilter:checked").map(function () {
    return $(this).val();
}).get();
displayProducts(selectedCategories);
});

// Show details
$(document).on("click", ".detailsButton", function () {
    $(this).next(".productDetails").slideToggle();
});
});
</script>
```

OUTPUT (BEFORE SELECTING CATEGORY):

Product Catalog

☐ Electronics ☐ Clothing

 Laptop High-performance laptop Price: \$800 Details	 Smartphone Latest smartphone Price: \$500 Details	 T-Shirt Comfortable cotton T-shirt Price: \$20 Details	 Jeans Stylish blue jeans Price: \$40 Details
--	--	---	---

OUTPUT (SELECTING CATEGORY):

Product Catalog

☒ Electronics ☐ Clothing

 Laptop High-performance laptop Price: \$800 Details	 Smartphone Latest smartphone Price: \$500 Details
--	--

LAB TASK

SOURCE CODE:

```
import React, { useState } from 'react';

import './App.css';

const students = [
  { name: "Alice", age: 20, skills: ["JavaScript", "React", "CSS"] },
  { name: "Bob", age: 22, skills: ["Java", "Spring Boot", "SQL"] },
  { name: "Charlie", age: 21, skills: ["Python", "Django", "Data Analysis"]
},
];

const App = () => {
  return (
    <div>
      <h1>Student Information App</h1>
      <StudentList />
    </div>
  );
};

const StudentList = () => {
  const [showMore, setShowMore] =
  useState(Array(students.length).fill(false));

  const toggleShowMore = (index) => {
    const updatedShowMore = [...showMore];
    updatedShowMore[index] = !updatedShowMore[index];
    setShowMore(updatedShowMore);
  };

  return (
    <div>
      {students.map((student, index) => (
        <div key={index} style={{ marginBottom: "20px" }}>
          <h3>{student.name}</h3>
          <p>Age: {student.age}</p>
          {showMore[index] && (
            <p>Skills: {student.skills.join(", ")}</p>
          )}
          <button onClick={() => toggleShowMore(index)}>
```

```
        {showMore[index] ? "Show Less" : "Show More"}  
      </button>  
    </div>  
  )}  
</div>  
);  
};  
  
export default App;
```

OUTPUT (WITHOUT CLICKING BUTTON):

Student Information App

Alice
Age: 20
Show More

Bob
Age: 22
Show More

Charlie
Age: 21
Show More

OUTPUT (AFTER CLICKING):

Student Information App

Alice
Age: 20
Skills: JavaScript, React, CSS
Show Less

Bob
Age: 22
Skills: Java, Spring Boot, SQL
Show Less

Charlie
Age: 21
Skills: Python, Django, Data Analysis
Show Less

HOME TASK:**SOURCE CODE:**

```
import React, { useState } from "react";

const products = [
  {
    name: "Laptop",
    price: "$1000",
    description: "High-performance laptop",
    details: "16GB RAM, 1TB SSD, Intel i7 Processor",
    reviews: ["Excellent product!", "Very fast and reliable."],
  },
  {
    name: "Phone",
    price: "$500",
    description: "Smartphone with a great camera",
    details: "128GB Storage, 48MP Camera, 5G",
    reviews: ["Great camera quality.", "Battery life could be better."],
  },
];

const App = () => {
  return (
    <div>
      <h1>Product Catalog</h1>
      <ProductList />
    </div>
  );
};

const ProductList = () => {
  const [showMore, setShowMore] =
    useState(Array(products.length).fill(false));

  const toggleShowMore = (index) => {
    const updatedShowMore = [...showMore];
    updatedShowMore[index] = !updatedShowMore[index];
    setShowMore(updatedShowMore);
  };

  return (
```

```
<div style={{ display: "grid", gridTemplateColumns: "1fr 1fr",
gap: "20px" }}>
  {products.map((product, index) => (
    <div key={index} style={{ border: "1px solid #ddd", padding:
"20px" }}>
      <h3>{product.name}</h3>
      <p>Price: {product.price}</p>
      <p>{product.description}</p>
      {showMore[index] && (
        <div>
          <p><strong>Details:</strong> {product.details}</p>
          <p><strong>Reviews:</strong> {product.reviews.join(" |
")}</p>
        </div>
      )}
      <button onClick={() => toggleShowMore(index)}>
        {showMore[index] ? "Show Less" : "Show More"}
      </button>
    </div>
  ))}
</div>
);
};

export default App;
```

OUTPUT (BEFORE CLICKING):

Product Catalog

Laptop

Price: \$1000

High-performance laptop

Show More

Phone

Price: \$500

Smartphone with a great camera

Show More

OUTPUT (AFTER CLIKCING):

Product Catalog

Laptop

Price: \$1000

High-performance laptop

Details: 16GB RAM, 1TB SSD, Intel i7 Processor

Reviews: Excellent product! | Very fast and reliable.

Show Less

Phone

Price: \$500

Smartphone with a great camera

Details: 128GB Storage, 48MP Camera, 5G

Reviews: Great camera quality. | Battery life could be better.

Show Less

LAB TASK:

SOURCE CODE:

```
using Microsoft.AspNetCore.Mvc;
using System.Collections.Generic;
namespace WebApplication1.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class ProductCatalogController : ControllerBase
    {
        // Static list to simulate a product catalog
        private static List<Product> products = new List<Product>
        {
            new Product { Id = 1, Name = "Laptop", Price = 1000, Description = "High-performance laptop" },
            new Product { Id = 2, Name = "Phone", Price = 500, Description = "Smartphone with a great camera" }
        };

        // GET api/productcatalog
        [HttpGet]
        public ActionResult<List<Product>> Get()
        {
            return Ok(products);
        }

        // POST api/productcatalog
        [HttpPost]
        public ActionResult<Product> Post([FromBody] Product product)
        {
            if (string.IsNullOrEmpty(product.Name) || product.Price <= 0)
            {
                return BadRequest("Product name and price are required.");
            }

            products.Add(product);
            return CreatedAtAction(nameof(Get), new { id = product.Id }, product);
        }
    }

    public class Product
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public double Price { get; set; }
        public string Description { get; set; }
    }
}
```

OUTPUT:

Responses

Curl

```
curl -X POST \
  'https://localhost:7124/api/ProductCatalog' \
  -H 'accept: text/plain' \
```

Server response

Code	Details
------	---------

201

Undocumented

Response body

```
{
  "id": 66,
  "name": "JEANS",
  "price": 15,
  "description": "BLACK JEANS"
}
```

Response headers

```
content-type: application/json; charset=utf-8
date: Sun, 22 Dec 2024 14:11:07 GMT
location: https://localhost:7124/api/ProductCatalog?id=66
server: Kestrel
```

Server response

Code	Details
------	---------

200

Response body

```
[
  {
    "id": 1,
    "name": "Laptop",
    "price": 1000,
    "description": "High-performance laptop"
  },
  {
    "id": 2,
    "name": "Phone",
    "price": 500,
    "description": "Smartphone with a great camera"
  },
  {
    "id": 66,
    "name": "JEANS",
    "price": 15,
    "description": "BLACK JEANS"
  }
]
```

HOME TASK:**1. HTTP Methods: Definitions and Use Cases**

HTTP methods are used to perform various operations on resources in a web application. Here are the main HTTP methods:

- **GET:**
Retrieves data from the server.
Use case: Fetching a list of products or details of a single product.
Example:

```
Csharp

[HttpGet]
public IEnumerable<Product> GetProducts() { ... }
```

- **POST:**
Sends data to the server to create a new resource.
Use case: Adding a new product to the inventory.
Example:

```
csharp

[HttpPost]
public IActionResult AddProduct([FromBody] Product product) { ...
}
```

- **PUT:**
Updates an existing resource on the server.
Use case: Updating the details of an existing product.
Example:

```
csharp

[HttpPut("{id}")]
public IActionResult UpdateProduct(int id, [FromBody] Product
product) { ... }
```

- **DELETE:**
Deletes a resource on the server.
Use case: Removing a product from the database.
Example:

```
csharp

[HttpDelete("{id}")]
public IActionResult DeleteProduct(int id) { ... }
```

- **PATCH:**
Partially updates a resource.

Use case: Modifying only the price of a product.

Example:

```
csharp

[HttpPatch("{id}")]
public IActionResult UpdatePrice(int id, [FromBody] decimal
price) { ... }
```

- **HEAD:**
Similar to GET, but retrieves only headers and not the body.
Use case: Checking if a resource exists without downloading it.
- **OPTIONS:**
Describes the communication options for the target resource.
Use case: Checking the allowed HTTP methods for a particular endpoint.

2. Purpose of [ApiController] and [Route] Attributes in WebAPI

- **[ApiController] Attribute:**
 - Marks the class as a WebAPI controller, enabling WebAPI-specific behaviors.
 - Features:
 - Automatic model validation: Returns 400 Bad Request if model validation fails.
 - Simplifies parameter binding: Automatically binds route data, query parameters, and request body to method parameters.
 - Assumes JSON input/output by default.

Example:

```
csharp

[ApiController]
public class ProductsController : ControllerBase { ... }
```

- **[Route] Attribute:**
 - Defines the route template for a controller or action.
 - Supports placeholders for dynamic values (e.g., {id}).

Example:

```
csharp

[Route("api/[controller]")]
```

```
public class ProductsController : ControllerBase { ... }
```

3. Default JSON Serialization in WebAPI

- **JSON Serialization:** WebAPI uses **System.Text.Json** (or **Newtonsoft.Json** in older projects) for serializing and deserializing JSON data.
- **Key Features:**
 - Converts objects to JSON and vice versa.
 - Handles cyclic references gracefully with configuration.
 - Allows customization of property naming conventions (e.g., camelCase).

Example:

```
csharp
services.AddControllers().AddJsonOptions(options =>
{
    options.JsonSerializerOptions.PropertyNamingPolicy =
JsonNamingPolicy.CamelCase;
});
```

LAB 11

TASK 1:

Source Code (STUDENT CONTROLLER):

```
using Microsoft.AspNetCore.Mvc;
using WebApplication2.Models;
using Microsoft.EntityFrameworkCore;

[ApiController]
[Route("api/[controller]")]
public class StudentsController : ControllerBase
{
    private readonly LabDatabaseContext _context;

    public StudentsController(LabDatabaseContext context)
    {
        _context = context;
    }

    [HttpGet]
    public async Task<ActionResult<IEnumerable<Student>>> GetStudents()
    {
        var students = await _context.Students.ToListAsync();
        return Ok(students);
    }

    [HttpGet("{id}")]
    public async Task<ActionResult<Student>> GetStudent(int id)
    {
        var student = await _context.Students.FindAsync(id);

        if (student == null)
        {
            return NotFound();
        }

        return Ok(student);
    }

    // POST: api/students
    [HttpPost]
    public async Task<ActionResult<Student>> PostStudent(Student student)
    {
        // Check if the model is valid
        if (!ModelState.IsValid)
        {
            return BadRequest(ModelState);
        }

        // Add the student to the database
        _context.Students.Add(student);
        await _context.SaveChangesAsync();

        // Return the created student with a 201 status code
        return CreatedAtAction(nameof(GetStudent), new { id = student.StudentId }, student);
    }
}
```

Source Code (STUDENT MODEL):

```
namespace WebApplication2.Models;
```

```
public class Student
{
    public int StudentId { get; set; }
    public string Name { get; set; }
    public int Age { get; set; }
}
```

Source Code (DATABASE CONNECTION):

```
using System;
using System.Collections.Generic;
using Microsoft.EntityFrameworkCore;

namespace WebApplication2.Models;

public partial class LabDatabaseContext : DbContext
{
    public LabDatabaseContext()
    {
    }

    public LabDatabaseContext(DbContextOptions<LabDatabaseContext> options)
        : base(options)
    {
    }

    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        => optionsBuilder.UseSqlServer("Server=DESKTOP-
TI3IL06\\SQLEXPRESS;Database=LabDatabase;Trusted_Connection=True;TrustServerCertificate=True
;");
    }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.OnModelCreatingPartial(modelBuilder);
    }

    partial void OnModelCreatingPartial(ModelBuilder modelBuilder);
    public virtual DbSet<Student> Students { get; set; }
}
```

SOURCE CODE(appsetting.js):

```
{
    "Logging": {
        "LogLevel": {
            "Default": "Information",
            "Microsoft.AspNetCore": "Warning"
        }
    }
}
```

```
},  
"AllowedHosts": "/*",  
"ConnectionStrings": {  
    "DefaultConnection": "Server=DESKTOP-TI3IL06\\SQLEXPRESS;Database=LabDatabase;Trusted_Connection=True;TrustServerCertificate=True;"  
}  
  
}
```

OUTPUT (GET):

```
[  
  {  
    "studentId": 1,  
    "name": "John Doe",  
    "age": 20  
  },  
  {  
    "studentId": 2,  
    "name": "Jane Smith",  
    "age": 22  
  },  
  {  
    "studentId": 3,  
    "name": "Alice Brown",  
    "age": 18  
  },  
  {  
    "studentId": 4,  
    "name": "string",  
    "age": 0  
  }  
]
```

OUTPUT (SET):

201

Undocumented

Response body

```
{  
  "studentId": 5,  
  "name": "Absar",  
  "age": 20  
}
```

Response headers

```
content-type: application/json; charset=utf-8  
date: Sun, 29 Dec 2024 06:18:20 GMT  
location: https://localhost:7231/api/Students/5  
server: Kestrel
```

TASK 2:

SOURCE CODE: (STUDENTCONTROLLER.cs):

```
using Microsoft.AspNetCore.Mvc;  
using WebApplication2.Models;  
using Microsoft.EntityFrameworkCore;
```

```
[ApiController]  
[Route("api/[controller]")]
```

```
public class StudentsController : ControllerBase
{
    private readonly LabDatabaseContext _context;

    public StudentsController(LabDatabaseContext context)
    {
        _context = context;
    }

    // GET: api/students
    [HttpGet]
    public async Task<ActionResult<IEnumerable<Student>>> GetStudents([FromQuery] int? age)
    {
        try
        {
            // LINQ Query: Filter students based on age if the query parameter is provided
            var students = _context.Students.AsQueryable();

            if (age.HasValue)
            {
                students = students.Where(s => s.Age > age.Value);
            }

            return Ok(await students.ToListAsync());
        }
        catch (Exception ex)
        {
            return StatusCode(500, $"Internal server error: {ex.Message}");
        }
    }

    // GET: api/students/{id}
    [HttpGet("{id}")]
    public async Task<ActionResult<Student>> GetStudent(int id)
    {
        var student = await _context.Students.FindAsync(id);

        if (student == null)
        {
            return NotFound();
        }

        return Ok(student);
    }

    // POST: api/students
    [HttpPost]
    public async Task<ActionResult<Student>> PostStudent(Student student)
    {
        // Check if the model is valid
        if (!ModelState.IsValid)
        {

```

```

        return BadRequest(ModelState);
    }

    // Add the student to the database
    _context.Students.Add(student);

    // Save changes to the database
    await _context.SaveChangesAsync();

    // Return the created student with a 201 status code
    return CreatedAtAction(nameof(GetStudent), new { id = student.StudentId }, student);
}
}

```

OUTPUT:

GET /api/Students

Parameters

Name	Description
age integer(\$int32) (query)	18

Execute Clear

Code Details

200

Response body

```

[
  {
    "studentId": 1,
    "name": "John Doe",
    "age": 20
  },
  {
    "studentId": 2,
    "name": "Jane Smith",
    "age": 22
  },
  {
    "studentId": 5,
    "name": "Absar",
    "age": 20
  },
  {
    "studentId": 7,
    "name": "Absar",
    "age": 20
  }
]

```

LAB 12

TASK 1:**SOURCE CODE (PRODUCTCONTROLLER.cs):**

```

using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using LabWebApi.Models; // Assuming you have a Models folder for Product entity

namespace LabWebApi.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class ProductsController : ControllerBase
    {

```

```
private readonly LabDatabaseContext _context;

public ProductsController(LabDatabaseContext context)
{
    _context = context;
}

// 1. Get all products
[HttpGet]
public async Task<IActionResult> GetProducts()
{
    var products = await _context.Products.ToListAsync();
    return Ok(products);
}

// 2. Get a product by ID
[HttpGet("{id}")]
public async Task<IActionResult> GetProduct(int id)
{
    var product = await _context.Products.FindAsync(id);

    if (product == null)
    {
        return NotFound();
    }

    return Ok(product);
}

// 3. Add a new product
[HttpPost]
public async Task<IActionResult> AddProduct(Product product)
{
    if (!ModelState.IsValid)
    {
        return BadRequest(ModelState);
    }

    _context.Products.Add(product);
    await _context.SaveChangesAsync();

    return CreatedAtAction(nameof(GetProduct), new { id = product.Id }, product);
}

// 4. Update an existing product
[HttpPut("{id}")]
public async Task<IActionResult> UpdateProduct(int id, Product product)
{
    if (id != product.Id)
    {

```

```
        return BadRequest("Product ID mismatch.");
    }

    _context.Entry(product).State = EntityState.Modified;

    try
    {
        await _context.SaveChangesAsync();
    }
    catch (DbUpdateConcurrencyException)
    {
        if (!_context.Products.Any(e => e.Id == id))
        {
            return NotFound();
        }
        else
        {
            throw;
        }
    }

    return NoContent(); // Return 204 (No Content) when successfully updated
}

// 5. Delete a product
[HttpDelete("{id}")]
public async Task<IActionResult> DeleteProduct(int id)
{
    var product = await _context.Products.FindAsync(id);
    if (product == null)
    {
        return NotFound();
    }

    _context.Products.Remove(product);
    await _context.SaveChangesAsync();

    return NoContent(); // Return 204 (No Content) after successful deletion
}
}
```

OUTPUT (PUT METHOD / UPDATE):

```
[
  {
    "id": 1,
    "name": "Mobile",
    "price": 200
  },
  {
    "id": 2
```

Request body

```
{  
  "id": 1,  
  "name": "SMARTPHONE",  
  "price": 200  
}
```

AFTER UPDATION:

OUTPUT (DELETE METHOD):

```
[
  {
    "id": 1,
    "name": "SMARTPHONE",
    "price": 200
  },
  {
    "id": 2,
    "name": "Blue Jeans",
    "price": 29.99
  },
  {
    "id": 3,
    "name": "T-Shirt",
    "price": 15.99
  },
  {
    "id": 4,
    "name": "Jacket",
    "price": 89.5
  },
  {
    "id": 5,
    "name": "Sneakers",
    "price": 59.99
  }
]
```

AFTER DELETION:

DELETE /api/Products/{id}

Parameters

Name	Description
------	-------------

id * required

integer(\$int32)
(path)

```
[
  {
    "id": 1,
    "name": "SMARTPHONE",
    "price": 200
  },
  {
    "id": 2,
    "name": "Blue Jeans",
    "price": 29.99
  },
  {
    "id": 3,
    "name": "T-Shirt",
    "price": 15.99
  },
  {
    "id": 4,
    "name": "Jacket",
    "price": 89.5
  },
  {
    "id": 5,
    "name": "Sneakers",
    "price": 59.99
  }
]
```

Review Questions

1. Explain how Web API endpoints are created and consumed.

How Web API endpoints are created and consumed:

🔗 Creating Endpoints:

- In ASP.NET Core, Web API endpoints are created by defining a controller class with the [ApiController] attribute and routing via [Route("api/[controller]")].
- HTTP methods like [HttpGet], [HttpPost], [HttpPut], and [HttpDelete] correspond to actions like fetching, creating, updating, and deleting data.

☐ Consuming Endpoints:

- You can consume Web API endpoints using tools like HttpClient in .NET, which allows sending HTTP requests and receiving responses asynchronously.

2. What is the advantage of using HttpClient for API consumption?

Advantages of using HttpClient for API consumption:

- Reusable: HttpClient should be reused for multiple requests to avoid issues like socket exhaustion.
- Asynchronous: Supports async calls, improving performance in I/O-bound operations.
- Customizable: Allows you to set headers, timeouts, and authentication for requests.
- Error Handling: Provides built-in methods to handle different response statuses.
- Content Serialization: Simplifies handling JSON or XML data and converting it into .NET objects.

1. Analyze the Database Connection Errors

Root Causes:

- **Network instability:** Intermittent network issues can lead to database connection failures.
- **Connection Pool Exhaustion:** If too many concurrent connections are made to the database, it may run out of available connections.
- **Timeouts:** Long-running queries or high database load may result in connection timeouts.
- **Database Configuration:** Improper configuration of connection strings (e.g., incorrect server names or credentials).

Solutions:

- **Implement Retry Policies:** Use libraries like `Polly` to implement retries with exponential backoff when a database connection fails.
- **Database Redundancy:** Set up database replication to provide failover mechanisms in case the primary database server becomes unavailable.
- **Connection String Configuration:** Ensure the connection string is correctly configured, including setting timeouts and connection pooling parameters. A sample configuration:
`Server=myServerAddress;Database=myDataBase;User Id=myUsername;Password=myPassword;Connect Timeout=30;`

Prevention Strategy:

- **Monitoring:** Set up automated monitoring for database connectivity.
 - **Testing:** Regularly test failover and redundancy setups.
 - **Capacity Planning:** Ensure the database can handle peak loads by scaling vertically or horizontally.
-

2. Address the Performance Degradation During Peak Hours

Root Causes:

- **High Traffic:** The application may become slow due to increased requests that the current infrastructure cannot handle.
- **Resource Saturation:** Lack of sufficient resources (CPU, memory, or database) can cause slower response times.
- **Inefficient Database Queries:** Poorly optimized queries can significantly impact performance, especially during high traffic.

Solutions:

- **Implement Caching:** Cache frequently accessed data to reduce the number of database queries. Use tools like Redis or in-memory caching.
- **Auto-Scaling:** Enable auto-scaling in cloud environments like Azure, which automatically adjusts resources based on demand.
- **Database Query Optimization:** Review and optimize queries, add indexes, and ensure proper query plans are in place.

Prevention Strategy:

- **Load Testing:** Regularly perform load testing to ensure the application can handle increased traffic.
 - **Proactive Scaling:** Set up auto-scaling rules to add resources before the application reaches its limits.
 - **Database Optimization:** Continuously optimize database queries and indexing strategies to improve response time.
-

3. Resolve the Incomplete Logs Issue

Root Causes:

- **Lack of Structured Logs:** Unstructured or inconsistent logs make it difficult to diagnose issues.
- **Missing Log Data:** Essential information such as error codes, timestamps, and user IDs might not be logged, making it difficult to trace issues.

Solutions:

- **Implement Structured Logging:** Use structured logging frameworks like Serilog to ensure logs are consistently formatted and easily queryable.
- `Log.Information("User {UserId} logged in at {Timestamp}", userId, DateTime.Now);`
- **Integrate with Azure Monitor or Application Insights:** These tools provide centralized log management and enable real-time monitoring.
- **Capture Essential Information:** Ensure logs capture critical data such as error messages, stack traces, user actions, and transaction details.

Prevention Strategy:

- **Centralized Logging:** Implement a centralized logging solution (e.g., ELK stack, Azure Monitor) to capture logs from all parts of the system.
- **Log Retention Policies:** Set up log retention policies to ensure important logs are retained for future analysis.

- **Proactive Monitoring:** Continuously monitor logs for anomalies using alerting systems in tools like Application Insights.
-

Step-by-Step Solutions:

1. Database Connection Errors:

- **Step 1:** Use retry policies (e.g., `Polly` library) to handle transient connection issues.
- **Step 2:** Configure a redundant database setup with failover.
- **Step 3:** Optimize the connection string and ensure proper timeout and pooling settings.

2. Performance Degradation:

- **Step 1:** Implement caching mechanisms for frequently requested data.
- **Step 2:** Enable auto-scaling for the application and database to handle high traffic.
- **Step 3:** Analyze and optimize database queries, including indexing and query restructuring.

3. Incomplete Logs:

- **Step 1:** Integrate structured logging frameworks like `Serilog` for consistent logging.
 - **Step 2:** Implement log aggregation tools like Azure Monitor or ELK Stack.
 - **Step 3:** Capture essential information and error details in logs, and ensure logs are accessible for analysis.
-

Preventive Measures:

- **Database:** Set up automated monitoring for database health and use a centralized logging system.
- **Performance:** Perform stress tests and ensure proactive scaling capabilities.
- **Logging:** Regularly audit logs and set up alerts for critical issues.

By following these steps, you will be able to resolve existing issues and ensure better application performance and monitoring in the future.